# Лабораторна робота № 1
# Робота з базовими типами даних

**Мета роботи**: ознайомитись з основними можливостями мови програмування Go, найпростішої структурою програми, з властивостями базових типів. Отримати практичні навички по роботі з базовими типами даних в мові Go.

## Короткі теоретичні відомості

*Тип даних* визначає множину значень, набір операцій, які можна застосовувати до таких значень, а також спосіб реалізації зберігання значень та виконання операцій.

У мові програмування Go розрізняють прості, складові та інші типи даних.

Прості типи даних можна розділити на:

– цілочисельний,
– дійсний,
– символьний,
– логічний.

Складові (складні) типи даних можна поділити на:

– комплексний,
– масиви,
– строки,
– зрізи,
– структури.

Також існують інші типи даних:

– покажчики;
– інтерфейси.

Коротко згадаємо основні типи даних.

*Цілочисельні* дані можуть бути представлені в знаковій і беззнаковій формі.

*Беззнакові цілі числа* (uint8, uint16, uint32, uint64, uint) представляються у вигляді послідовності бітів в діапазоні від 0 до $2^n$-1, де n-кількість розрядів.

*Знакові цілі числа* (int8, int16, int32, int64, int) представляються в діапазоні $-2^{n-1}$ ... $+2^{n-1}$-1. При цьому старший біт даних відводиться під знак числа (“0” відповідає позитивному числу, “1” – негативному).

*Дійсний тип* (float32, float64) призначений для представлення дійсних чисел. Дійсні числа представляються в розрядній сітці машини у нормованій формі. Нормована форма числа передбачає наявність однієї значущої цифри у цілій частини мантиси. Мантиса множиться на основу системи числення у відповідній степені. Наприклад, число 12345,678 в нормованій формі для десяткової системи числення можна представити як:

$12345{,}678 = 1{,}2345678 \cdot 10^4$.

Число 0,009876 в нормованій формі для десяткової системи числення можна представити як:

$0{,}009876 = 9{,}876 \cdot 10^{-3}$.

Представленні числа записані як числа з плаваючою комою в десятковій системі числення. Аналогічно представляються числа в комп’ютері, тільки використовується двійкова система числення.

*Логічний (булевий) тип* (bool) - це спеціальний однобітний цілочисельний тип, який використовується для представлення істинності та хибності. Булева змінна може приймати тільки два значення: *true*, *false*.

*Комплексні числа* (complex64, complex128) – це вираження виду $a+bi$, де $a$, $b$ – дійсні числа, а $i$ – так звана уявна одиниця, квадрат якого дорівнює $-1$. Число $a$ називається дійсною частиною, а число $b$ – уявною частиною комплексних чисел.

*Масив* – структура даних, яка зберігає впорядкований набір фіксованої кількості однотипних елементів, що зберігаються в послідовно розташованих комірках оперативної пам'яті, мають порядковий номер і спільне ім'я, що надає користувач.

*Символьний тип* (rune) – тип даних, призначений для зберігання одного символу (керуючого або друкованого) в певному кодуванні. Може бути як однобайтовим, так і мультибайтним. Символьні літерали визначаються за допомогою одинарних лапок (').

*Строковий тип* (string) – це тип даних, значеннями якого є довільна послідовність (рядок) символів алфавіту. Строки в мові Go є незмінними послідовностями довільних байтів.

Строкові літерали визначаються за допомогою подвійних лапок (") або зворотних апострофів (`). Лапки використовуються для визначення інтерпретованих строкових літералів – такі строки підтримують екрановані послідовності.

У мові програмування Go використовується кодування UTF-8. UTF-8 (від англ. Unicode Transformation Format, 8-bit – «формат перетворення Юнікоду, 8-бітний») – одна із загальноприйнятих і стандартизованих кодувань тексту, яка дозволяє зберігати символи Юнікоду, використовуючи *змінну кількість байт*. Стандарт UTF-8 офіційно закріплений в документах RFC 3629 і ISO/IEC 10646 Annex D.

Формат UTF-8 був розроблений у 1992 році Кеном Томпсоном і Робом Пайком. Перевагою кодування є сумісність з ASCII (7-бітові символи відображаються як є). Тому в разі, якщо латинські букви і найпростіші знаки пунктуації (включаючи пробіл) займають істотний обсяг тексту, UTF-8 дає виграш за обсягом у порівнянні з UTF-16

З іншими типами змінних ознайомимося у подальшому.

У загальному випадку оголошення змінної можна представити в наступному вигляді:

*var <ім'я> <тип> [= <ініціалізатор>]*

Наприклад:

```
var defaultInt32 int32 // Цілочисельна знакова змінна
var userInt uint32 = 10 // Цілочисельна беззнакова змінна
var defaultfloat32 float32 // Дійсна змінна
var third bool = true  //Булева змінна
var first complex64 = 0.37 + 1.72i //Комплексна змінна
```

```
var first string = "First: line 1\nline 2" // Строкова змінна
var second string = `Second: line 1\nline 2` // Строкова змінна
var letter rune = 'A' // Символьна змінна
var array [5]int =[5]int{1, 7, 4, 6, 3} // Масив типу [5]int
```

Змінні можна групувати, щоб не повторювати ключове слово "var" перед кожним оголошенням змінної.

```
var (
    defaultInt32 int32 // Цілочисельна знакова змінна
    userInt uint32 = 10 // Цілочисельна беззнакова змінна
    defaultfloat32 float32 // Дійсна змінна
    third bool = true  //Булева змінна
    first complex64 = 0.37 + 1.72i //Комплексна змінна
    first string = "First: line 1\nline 2" // Строкова змінна
    second string = `Second: line 1\nline 2` // Строкова змінна
    letter rune = 'A' // Символьна змінна
    array [5]int =[5]int{1, 7, 4, 6, 3} // Масив типу [5]int
)
```

Всі змінні, що не були ініційовані, ініціалізуються значенням за замовчуванням: 0 – для чисел; порожній рядок – для строкових змінних тощо.

Якщо змінну необхідно створити і ініціалізувати, то можна використовувати скорочений запис для оголошення змінних. Для цього використовується операція ініціалізації ": =".

Наприклад:

```
userInt := -10 // Цілочисельна знакова змінна
userfloat64:= -3.4 // Дійсна змінна
third := true  //Булева змінна
first := 0.37 + 1.72i //Комплексна змінна
first := "First: line 1\nline 2" // Строкова змінна
second := `Second: line 1\nline 2` // Строкова змінна
letter := 'A' // Символьна змінна
array :=[5]int{1, 7, 4, 6, 3} // Масив типу [5]int
```

Не допускається присвоювання будь-яких змінних різного типу один одному неявним способом.

*Константа* – спосіб адресації даних, зміна яких не передбачається або забороняється. Оголошуються константи за допомогою ключового слова “*const*”. Наприклад:

```
const limit = 512 // константа, сумісна з будь-якими числовими типами
const top uint16 = 1421 // константа типу uint16
```

У випадках, коли потрібно лише оголосити константи з відмінними значеннями, можна скористатися підтримкою *перерахувань*.

Наприклад:

```
const (
    Cyan = 0
    Magenta = 1
    Yellow = 2
)
```

або

```
const (
    Cyan = iota // 0
    Magenta     // 1
    Yellow      // 2
)
```

Ключове слово "*iota*" представляє послідовні цілочисельні константи, які вона приймає для кожного рядка перерахування.

Структура програми в загальному випадку має наступний вигляд:

```
package main

//Імпорт пакетів
import "fmt"

func main() {  // Головна функція
//Оголошення локальної змінної
    var str string = "Golang!"
// Виведення в консоль тексту
    fmt.Println("Hello ", str)
}
```

Програма на мові Go зберігається в одному або декількох файлах. Кожен файл з програмним кодом повинен належати якомусь пакету, тому спочатку кожного файлу повинно бути оголошення пакета, до якого цей файл належить. Пакет оголошується за допомогою ключового слова *package*. Причому головний пакет програми повинен називатися "main".

У файлі може використовуватися функціонал з інших пакетів. В цьому випадку пакети треба імпортувати за допомогою ключового слова *import*.

Після підключення пакетів розташовуються оголошення типів, змінних, функцій, констант.

Вхідною точкою в програмі є функція з ім'ям “main”.

Далі в функції main розташовуються оголошення локальних змінних та констант, після чого йдуть інструкції програмного коду у відповідності з алгоритмом програми.

*fmt.Println("Hello ", str)* – це виклик функцій *Println()* з пакету *fmt*. Дана функція призначена для виведення інформації у консоль. Існують і інші функції в пакеті *fmt*.

Приклад більш складної програми на мові Go:

```
package main

import "fmt"

func main() {
    var a, b int

    fmt.Printf("a=")
    n, err := fmt.Scanf("%d\n", &a)
    if err != nil || n != 1 {
      fmt.Println(n, err)
  }

    fmt.Printf("b=")
    n, err = fmt.Scanf("%d\n", &b)
    if err != nil || n != 1 {
      fmt.Println(n, err)
    }
    fmt.Printf("a+b=%d", a + b)
}
```

## Порядок виконання роботи

1. Ознайомитися з прикладами кодів;

2. Виконати завдання, зазначені в прикладах;

3. Підготувати звіт. Структура звіту: номер лабораторної роботи, назва лабораторної роботи, мета роботи, хід роботи, висновки по роботі.

**Контрольні питання**

1. Перерахуйте типи даних та поясніть їх основні властивості.
2. Яка структура програми на мові Go?
3. Які існують цілочисельні типи даних?
4. Яка різниця між цілочисельні типами даних?
5. Як зберігаються строки в мові Go?